

Lab 3: Simple Linear Regression

Part A: Data Collection and Preprocessing

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import pickle
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# 1. Load the dataset using Pandas
# Assuming the file is 'data/Department Awareness Survey (Responses).xlsx'
df = pd.read_excel('data/Department Awareness Survey (Responses).xlsx')
```

```
In [2]: # 2. Display the first 5 rows
df.head()
```

Out[2]:

	Timestamp	Registration Number	Email	Job role that you are interested in
0	2026-06-22 08:49:31.362	2547123	jiyaelza.jabi@mca.christuniversity.in	Software Engineer
1	2026-06-22 08:49:50.520	2547122	jinishaleema.rosario@mca.christuniversity.in	Data Engineer
2	2026-06-22 08:50:44.521	2547101	rajeev.chandar@mca.christuniversity.in	Software Engineer
3	2026-06-22 08:51:00.666	2547156	sounak.chakraborty@mca.christuniversity.in	Data Scientist
4	2026-06-22 08:51:42.357	2547148	samar.subhash@mca.christuniversity.in	Software Engineer

```
In [3]: # 3. Check dataset dimensions
print("Dimensions:", df.shape)
```

Dimensions: (54, 15)

```
In [4]: # Save to CSV as requested
df.to_csv('data/dataset.csv', index=False)
print("Saved to CSV.")
```

Saved to CSV.

```
In [5]: # 4. Identify missing values
print("Missing values per column:\n", df.isnull().sum())
```

Missing values per column:

Timestamp

0

Registration Number

0

Email

0

Job role that you are interested in\n

0

What is the minimum salary of students placed through campus (In LPA..respond as a number) 0

What is the maximum salary of students placed through campus (In LPA..respond as a number) 0

What is the median salary of students placed through campus (In LPA..respond as a number) 0

Which is the highest paying company

0

Rate your contribution towards extra curricular activities

0

Rate your technical competencies

0

What are your package expectations (LPA)

0

your CIA % of last semester

0

your GPA of last semester

0

Your maximum attendance % till last semester

0

Internships Interests

0

dtype: int64

```
In [6]: # Rename columns for easier access
df.rename(columns={
    'your CIA % of last semester ': 'CIA',
    'your GPA of last semester': 'GPA',
    'Your maximum attendance % till last semester': 'Attendance'
}, inplace=True)
```

```
In [7]: # 5. Remove or handle null values
# 6. Convert required columns into numerical datatype
# The Attendance column has string 'Option 1', we should coerce to numerical
df['Attendance'] = pd.to_numeric(df['Attendance'], errors='coerce')
df['CIA'] = pd.to_numeric(df['CIA'], errors='coerce')
df['GPA'] = pd.to_numeric(df['GPA'], errors='coerce')

# Drop nulls in the required columns
df_clean = df.dropna(subset=['CIA', 'Attendance', 'GPA']).copy()
print("Dimensions after dropping nulls:", df_clean.shape)
```

Dimensions after dropping nulls: (46, 15)

```
In [8]: # 7. Remove duplicate records if present
df_clean.drop_duplicates(inplace=True)
print("Dimensions after dropping duplicates:", df_clean.shape)
```

Dimensions after dropping duplicates: (46, 15)

```
In [9]: # 8. Generate statistical summary
```

```
df_clean[['CIA', 'Attendance', 'GPA']].describe()
```

Out [9]:

	CIA	Attendance	GPA
count	46.000000	46.000000	46.000000
mean	58.326763	79.100402	5.623913
std	30.939425	33.757066	12.769951
min	0.630000	0.870000	2.900000
25%	65.002500	86.250000	3.400000
50%	70.000000	92.000000	3.505000
75%	78.000000	96.750000	3.730000
max	89.000000	100.000000	90.000000

```
In [10]: # 9. Select appropriate dependent and independent variables
# Experiment 1
X_cia = df_clean[['CIA']]
Y_gpa = df_clean['GPA']

# Experiment 2
X_att = df_clean[['Attendance']]
Y_gpa_att = df_clean['GPA']
```

Part B: Simple Linear Regression using Scikit-learn

```
In [11]: # Experiment 1: CIA vs GPA
X_train_cia, X_test_cia, Y_train_cia, Y_test_cia = train_test_split(X_cia, Y_gpa,
                                                                    test_size=0.2,
                                                                    random_state=42)
model_cia = LinearRegression()
model_cia.fit(X_train_cia, Y_train_cia)

slope_cia = model_cia.coef_[0]
intercept_cia = model_cia.intercept_
print(f"Scikit-learn - CIA vs GPA: Slope = {slope_cia:.4f}, Intercept = {intercept_cia:.4f}")

preds_cia_sk = model_cia.predict(X_test_cia)
print("Predictions (Scikit-learn):", preds_cia_sk)
```

```
Scikit-learn - CIA vs GPA: Slope = 0.0687, Intercept = 1.6579
Predictions (Scikit-learn): [1.93260507 1.71078474 7.1519069  7.56395704
6.46515666 7.01455685
6.46515666 1.70673978 6.46515666 1.89483381]
```

```
In [12]: # Experiment 2: Attendance vs GPA
X_train_att, X_test_att, Y_train_att, Y_test_att = train_test_split(X_att, Y_gpa_att,
                                                                    test_size=0.2,
                                                                    random_state=42)
model_att = LinearRegression()
model_att.fit(X_train_att, Y_train_att)

slope_att = model_att.coef_[0]
intercept_att = model_att.intercept_
print(f"Scikit-learn - Attendance vs GPA: Slope = {slope_att:.4f}, Intercept = {intercept_att:.4f}")

preds_att_sk = model_att.predict(X_test_att)
print("Predictions (Scikit-learn):", preds_att_sk)
```

Scikit-learn – Attendance vs GPA: Slope = 0.0113, Intercept = 4.9843
 Predictions (Scikit-learn): [6.11618218 4.99523214 6.07090499 5.95771201
 5.9916699 6.10871144
 5.9916699 4.99492086 6.08222429 6.0029892]

Part C: Manual Computation using Ordinary Least Squares (OLS)

```
In [13]: # Experiment 1 Manual OLS
def manual_ols(X, Y):
    X_mean = np.mean(X)
    Y_mean = np.mean(Y)

    numerator = np.sum((X - X_mean) * (Y - Y_mean))
    denominator = np.sum((X - X_mean) ** 2)

    slope = numerator / denominator
    intercept = Y_mean - slope * X_mean
    return slope, intercept

# Using training data for consistency
slope_cia_manual, intercept_cia_manual = manual_ols(X_train_cia['CIA'], Y
print(f"Manual OLS – CIA vs GPA: Slope = {slope_cia_manual:.4f}, Intercept

preds_cia_manual = slope_cia_manual * X_test_cia['CIA'] + intercept_cia_m
print("Predictions (Manual):", preds_cia_manual.values)
```

Manual OLS – CIA vs GPA: Slope = 0.0687, Intercept = 1.6579
 Predictions (Manual): [1.93260507 1.71078474 7.1519069 7.56395704 6.46515
 666 7.01455685
 6.46515666 1.70673978 6.46515666 1.89483381]

```
In [14]: # Experiment 2 Manual OLS
slope_att_manual, intercept_att_manual = manual_ols(X_train_att['Attendan
print(f"Manual OLS – Attendance vs GPA: Slope = {slope_att_manual:.4f}, I

preds_att_manual = slope_att_manual * X_test_att['Attendance'] + intercep
print("Predictions (Manual):", preds_att_manual.values)
```

Manual OLS – Attendance vs GPA: Slope = 0.0113, Intercept = 4.9843
 Predictions (Manual): [6.11618218 4.99523214 6.07090499 5.95771201 5.99166
 99 6.10871144
 5.9916699 4.99492086 6.08222429 6.0029892]

Comparison Task

```
In [15]: diff_cia = preds_cia_sk - preds_cia_manual.values
diff_att = preds_att_sk - preds_att_manual.values
print("Max absolute difference for CIA vs GPA:", np.max(np.abs(diff_cia))
print("Max absolute difference for Attendance vs GPA:", np.max(np.abs(dif

print("Observation: Both methods produce identical predictions (up to flo
```

Max absolute difference for CIA vs GPA: 0.0
 Max absolute difference for Attendance vs GPA: 8.881784197001252e-16
 Observation: Both methods produce identical predictions (up to floating po
 int precision) since Scikit-learn Linear Regression uses the OLS formulati
 on under the hood.

Parameter Saving Task

```
In [16]: # Save weights for Experiment 1 (CIA vs GPA) as an example
weights = {
    'slope': slope_cia,
    'intercept': intercept_cia
}

with open('linear_regression_weights.pkl', 'wb') as f:
    pickle.dump(weights, f)
print("Saved parameters to linear_regression_weights.pkl")
```

Saved parameters to linear_regression_weights.pkl

```
In [17]: # Load weights
with open('linear_regression_weights.pkl', 'rb') as f:
    loaded_weights = pickle.load(f)
print("Loaded parameters:", loaded_weights)

# Predict using loaded parameters
sample_X = X_test_cia['CIA'].iloc[0]
pred_loaded = loaded_weights['slope'] * sample_X + loaded_weights['intercept']
print(f"Prediction for X={sample_X} using loaded weights: {pred_loaded:.4f}")
```

Loaded parameters: {'slope': np.float64(0.06867502407800606), 'intercept': np.float64(1.657904973456076)}

Prediction for X=4.0 using loaded weights: 1.9326